# Paid, but nothing arrived.

When a customer says it, this agent finds where the money got stuck — with proof — and drafts the fix.

1 · THE CUSTOMER WRITES

**“I paid 24,900 won but my credits never came.”**

Just the message and their email.
No order number needed.

→

2 · ONE SQL DOOR READS THE SELLER'S OWN RECORDS **ZETARIS**

- 💳 Payment provider ledger
- 🧾 Orders database
- 🎟️ Credit grants
- 📜 Server logs · ✉️ Mail events

Read-only. Nothing is copied out of the seller's systems.

→

3 · THE AGENT INVESTIGATES **NVIDIA NEMOTRON** **VIA METERLESS NIGHTSHIFT**

The model picks what to look up next: *no order under this email → who paid at the provider? → which order is that payment? → full history*.
Emails and card digits are masked before the model sees any row.

4 · A RULE-BASED VERIFIER CHECKS THE MONEY CHAIN — THE MODEL NEVER DECIDES

**Card authorized** › **Provider says PAID** › **Order confirmed** › **Credits granted ✕** went to a typo’d address › **Mail delivered ✕** bounced

Green = proven by a row · Red = the link that broke, with the rows that prove it · Grey = never reached (e.g. “you were never charged”).

5 · YOU GET

### Why it broke — with citations

The explanation must cite the real rows it used; if it cites anything else, it is replaced.

5 · YOU GET

### A reply to send

In the customer’s language. It can’t promise refunds — a person decides money.

6 · YOU APPROVE

### The fix runs, the chain turns green

Re-grant, resend, move credits — only after you click Approve. Every fix is logged.

**Who it’s for:** small sellers of digital goods (credits, e-books, subscriptions) with no support team.

**Why it matters:** a slow answer becomes a card dispute. The customer already has proof they paid; this finds yours in one pass.

**Built from real incidents:** the cases are bugs we hit in our own shop, rebuilt with synthetic data.